3D Data Science with Python

Sergio Peral · ML Engineer, Teqyard



3D meant C++; now Python drives it

C++ with PCL

```
using Cloud =
    pcl::PointCloud<pcl::PointXYZ>;
Cloud::Ptr cloud(new Cloud);
Cloud down;
if (pcl::io::loadPCDFile(
    "crop.pcd", *cloud)) return 1;
pcl::VoxelGrid<pcl::PointXYZ> vg;
vg.setInputCloud(cloud);
vg.setLeafSize(0.5f, 0.5f, 0.5f);
vg.filter(down);
std::cout << cloud->size()
  << " -> " << down.size() << "\n";


# output
722961 -> 230519
```

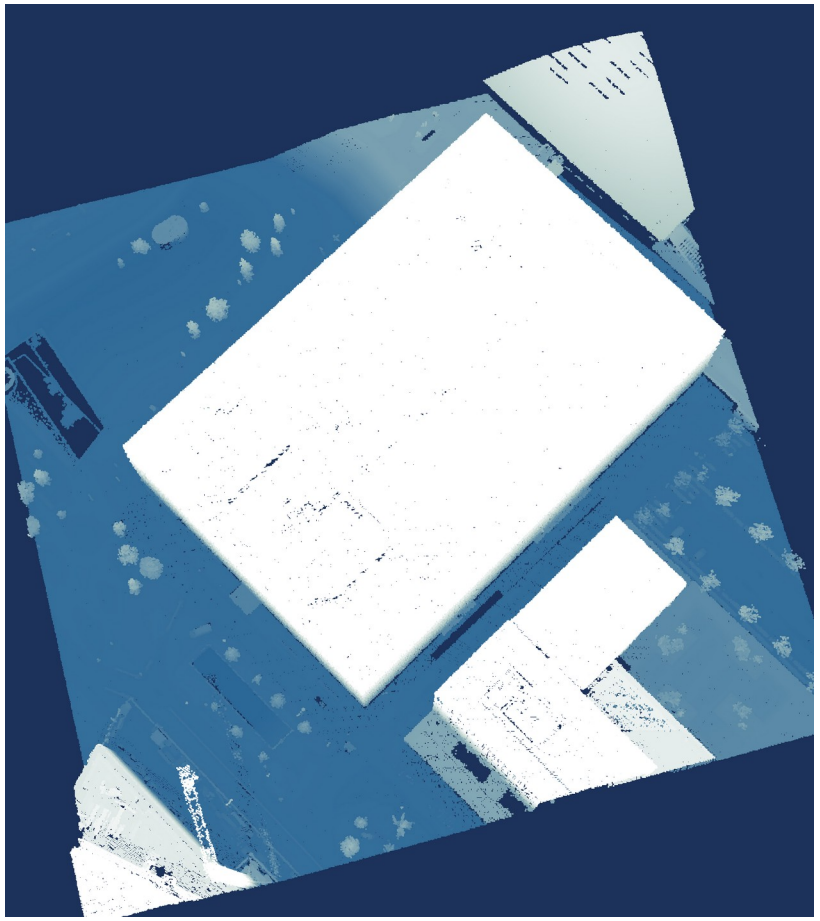
Python with Open3D

```
pcd = o3d.io.read_point_cloud(path)
down = pcd.voxel_down_sample(0.5)
print(len(pcd.points), "->",
      len(down.points))


# output
722961 -> 232180
```

Why do the counts differ? PCL and Open3D anchor the 0.5 m voxel grid differently; both average the points in each voxel.

A city scan: millions of points, no names



722 961 points in one city block; none has a name.

- Each point: x, y, z and little else
- Cities register what stands in the street: trees, lamp posts, bollards
- Naming points by hand doesn't scale

Today's question: does the scan agree with the register?

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

The goal: name every point, then check it

From x, y, z alone, in plain Python, and say how far to trust it.

In

- One LAZ file, 200 m × 200 m
- x, y, z per laser echo

Out

- A class per point: ground, buildings, vegetation, street furniture

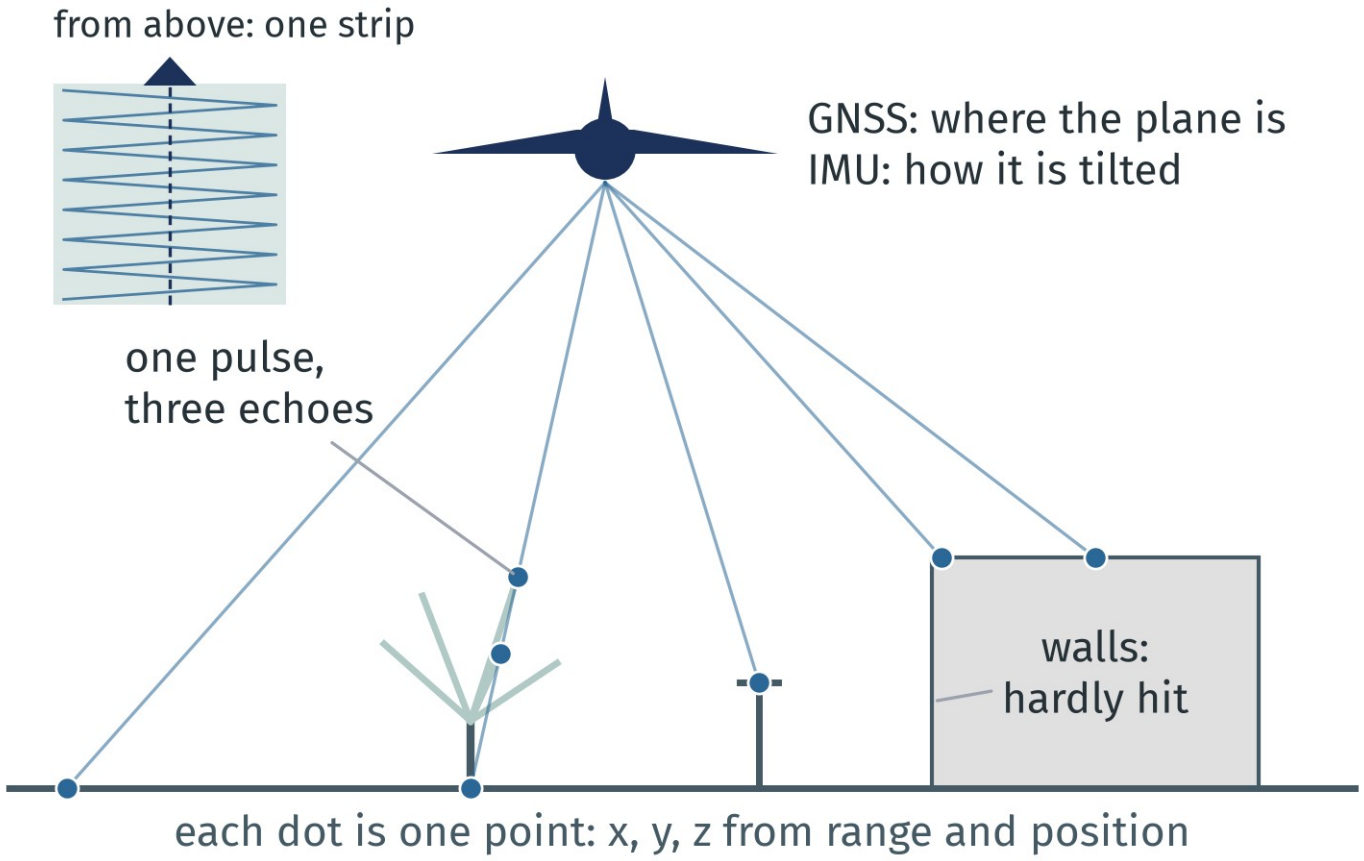
Checked against

- Ground, buildings: AHN's own classes
- Poles: the BGT register
- Vegetation: no reference

Eight chapters, three questions

Chapter	What you get	Rule	Min
Intro	The problem, the goal, this map		3
Data	How a plane scans; the case; the scope		3
Toolbox	Four rules; a library for every layer		2
Pipeline	Six steps from LAZ file to labels	1, 2	8
Memory	Does it fit? A full AHN sheet	2	2
Speed	Is it fast? A loop against one call	3	5
Reality	Is it right? The register against the scan	4	1
Wrap-up	The rules in numbers; limits; next		3

A plane sweeps a strip with a laser

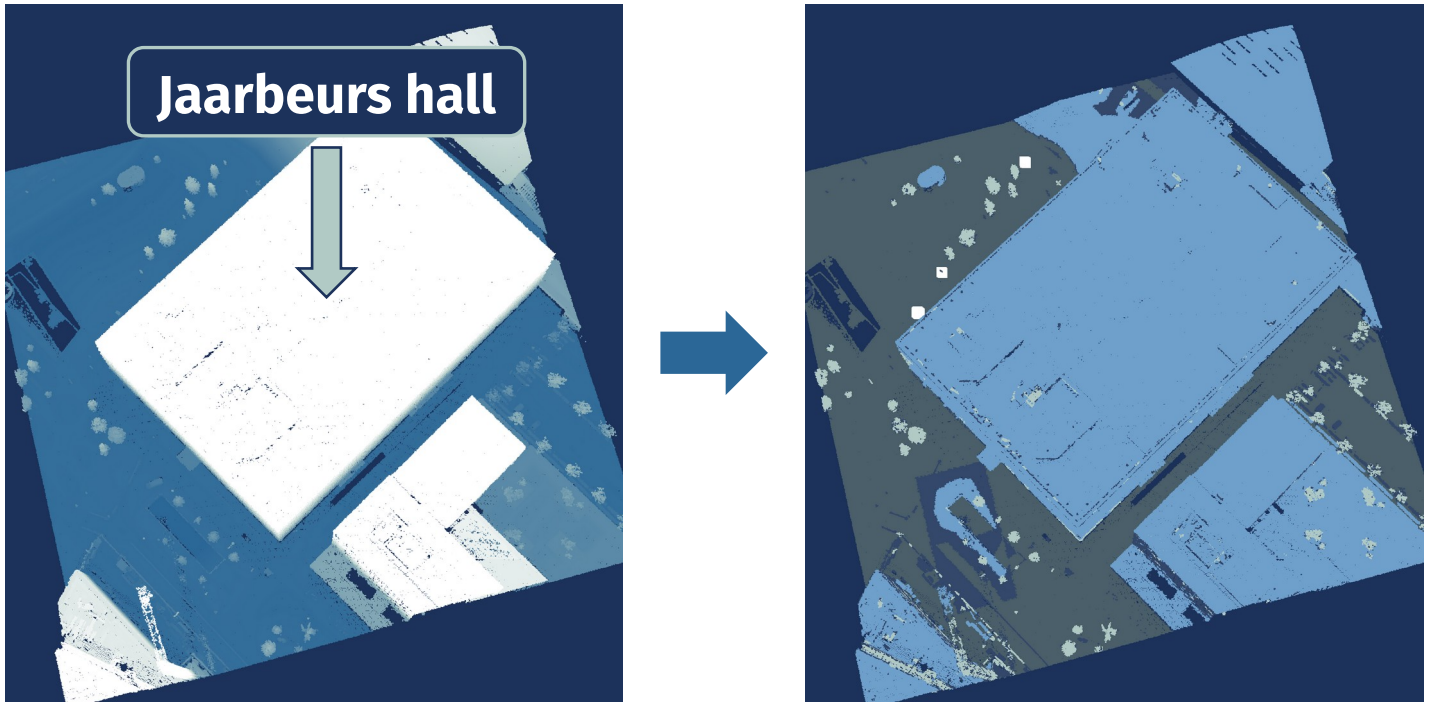


Schematic side view, inset from above; not to scale.

x, y, z = range + GNSS position + IMU tilt.

- A scanning laser, not one spot: it sweeps a strip
- One pulse can leave up to 7 echoes, in trees most
- Roofs dense, walls sparse, thin poles often missed
- This crop: 691 011 pulses in 19.8 s of flight

Today's case: the block around the Jaarbeurs



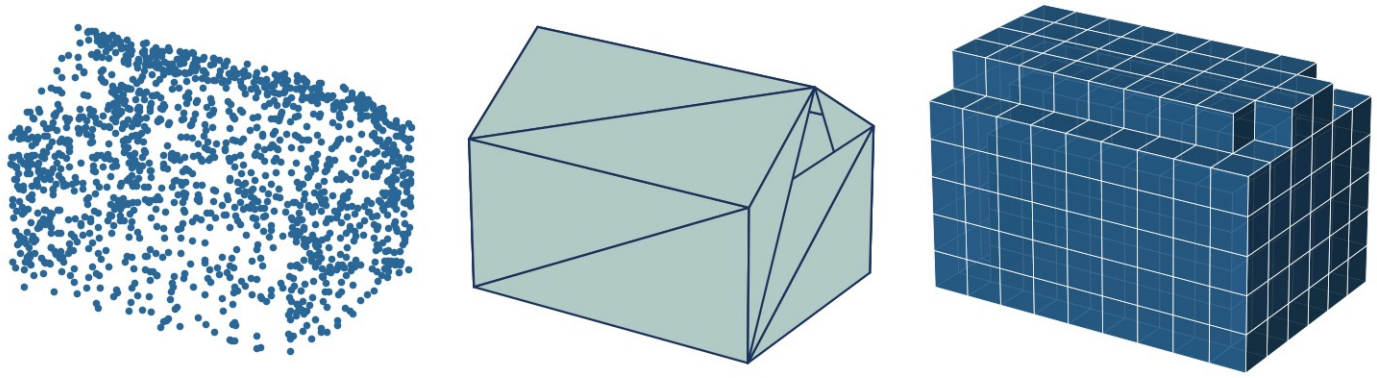
The data set

Area	200 m × 200 m
Points	722 961
Density	about 18 per m²
Source	AHN5, ahn.nl
Flown	April 2023, night
File	LAZ, 4.3 MB
Coordinates	RD New + NAP
Licence	CC BY 4.0

Checked against AHN (ground, buildings) and the BGT pole register (street furniture).

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

One object, three representations



Point cloud
1500 × 3 floats,
no connectivity

Mesh
10 vertices,
16 triangles

Voxel grid
182 cells of
0.5 m

Points measure, meshes connect, voxels index space. Convert only when the task needs it.

- Point cloud: an unordered N×3 array plus attributes; no neighbours stored.
- Mesh: vertices plus faces, so surfaces, normals and areas are defined.
- Voxel grid: occupied cells at one resolution; detail below a cell is gone.

What this talk does, and doesn't

In scope

- Airborne LiDAR, AHN5
- One 200 m × 200 m urban crop
- Four classes, rule-based
- Standard Python libraries
- Speed and memory

Not today

- Deep learning, trained models
- Mobile or terrestrial scans
- Meshes, registration
- CRS in depth, production
- State-of-the-art accuracy

Assumptions

- Metres: RD New + NAP
- Ground: one plane per 25 m tile
- Voxel 0.5 m; inliers 0.4 m; objects > 0.3 m
- DBSCAN eps 0.75 m, 6 points
- Thresholds tuned on this crop

Four rules this talk will prove

1 Think in arrays, not points

A point cloud is an N×3 table plus attribute columns.

Proof: Pipeline

2 Make it small first

Crop, tile and downsample; scale up last.

Proof: Pipeline, Memory

3 Let compiled code loop

One call per array or tile, never per point.

Proof: Speed

4 Look before you compute

Render it; check units, coordinates and classes.

Proof: Reality

Python already has a library for every layer

Load and store	laspy	PDAL	pyproj	
Arrays and speed	NumPy	SciPy	Numba	dask
Process	OPEN3D	trimesh		
Visualise	PyVista	VTK		
Learn	scikit-learn	PyTorch3D		

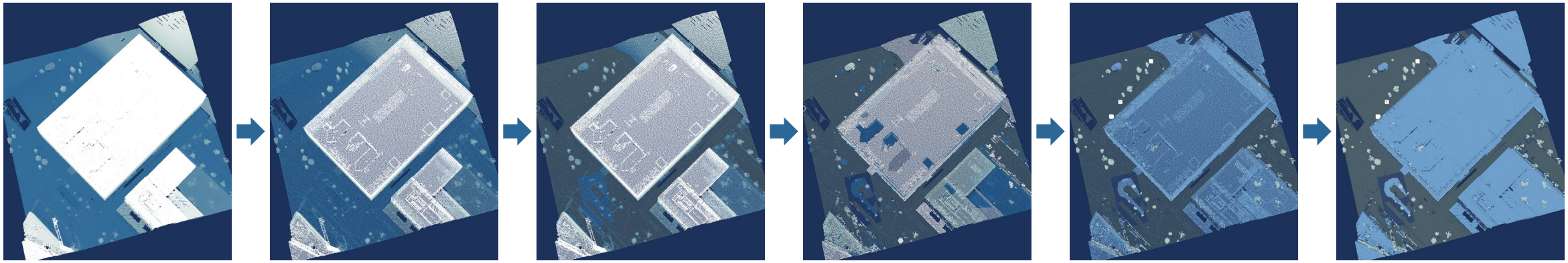
Python plans; compiled code does the work



Every call you make here runs compiled code

- NumPy: a compiled C core, with some C++
- Open3D: C++, wrapped for Python
- PyVista: VTK, which is C++
- laspy: the lazrs decoder is Rust

Six steps: from LAZ file to labels



Load	**Voxel downsampling**	**Ground removal**	**Clustering**	**Labelling**	**Paint-back**
laspy	Open3D	Open3D	Open3D, SciPy	NumPy, SciPy	SciPy, PyVista

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

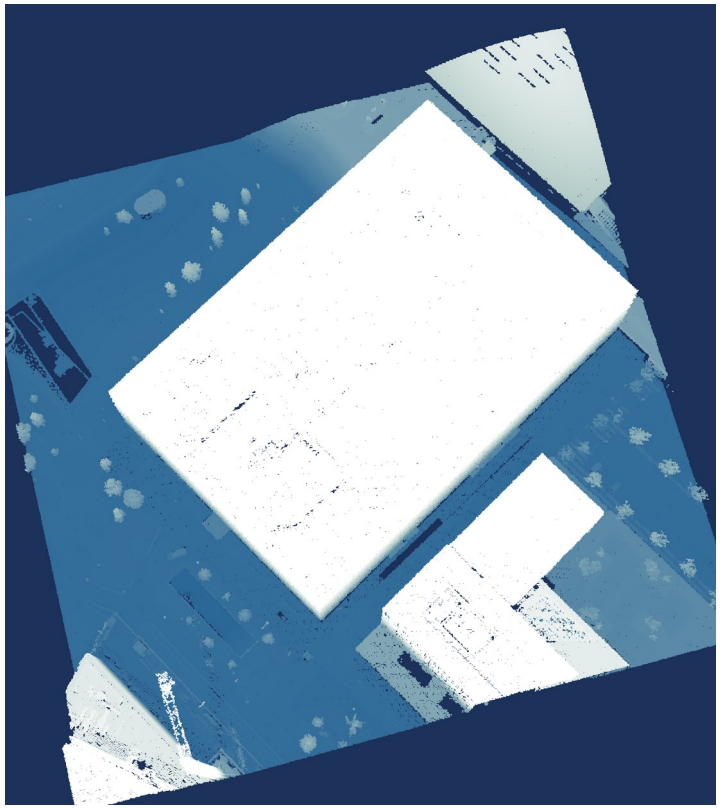
1 Load: subtract a local origin first

```python
import laspy
import open3d as o3d
las = laspy.read("crop.laz")
xyz = las.xyz - las.header.mins
v = o3d.utility.Vector3dVector(xyz)
pcd = o3d.geometry.PointCloud(v)
```



float32 step

1.6 cm →
after origin shift:
15 µm



laspy hands you NumPy; a local origin keeps float32 precise.

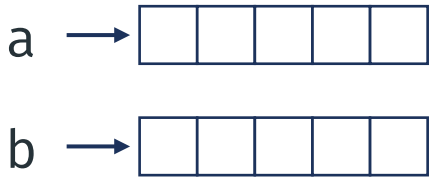
Data: AHN5, Het Waterschapshuis, CC BY 4.0.

1 Load: pass views, not copies

Why 24 B? x, y and z are three float64 values: 3 × 8 B.

Rule 1: each extra xyz copy costs 24 B per point → 0.5 GB per 1 km AHN file.

The problem: copies `copy`



new memory → costs time & RAM

The solution: views `view`



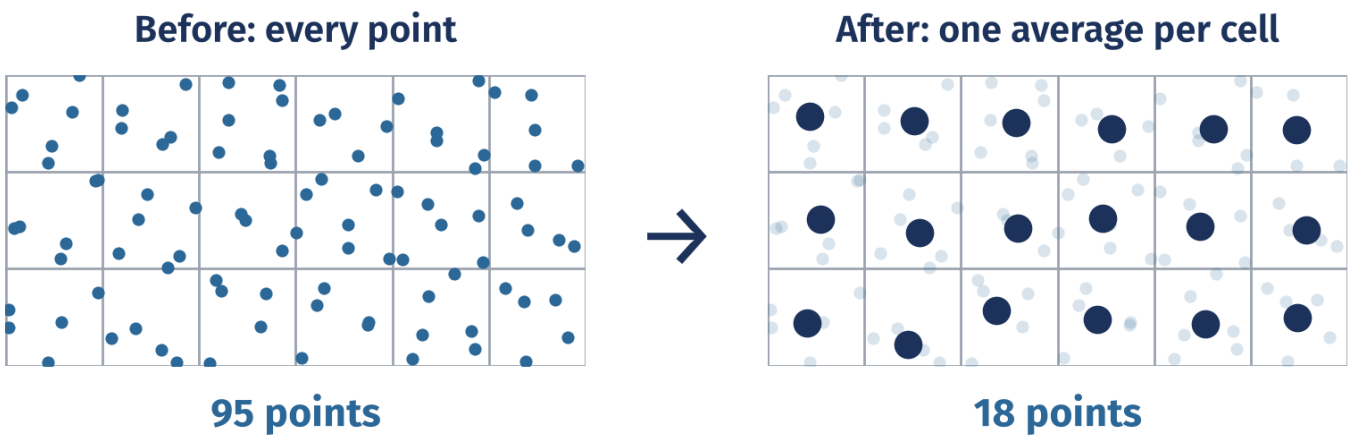
same memory → free, writes show through

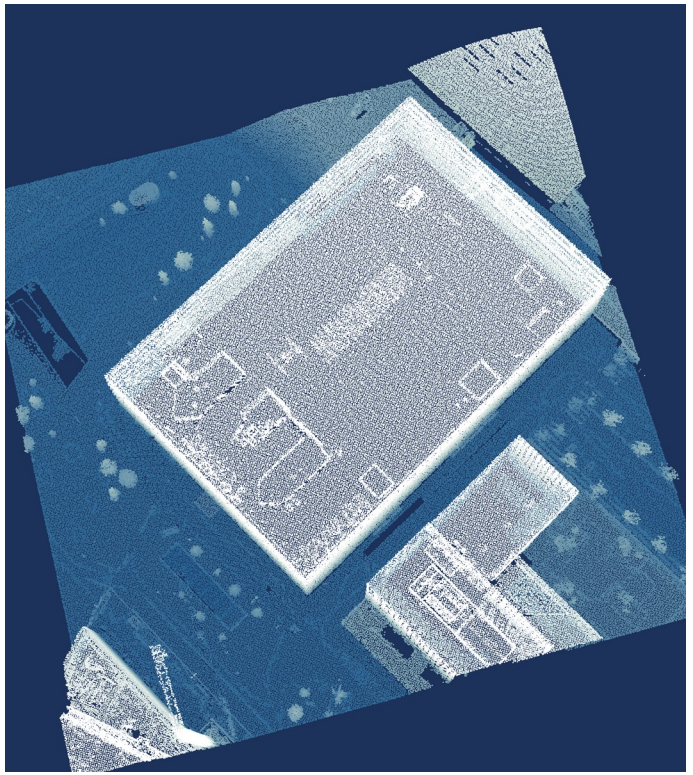
`las.xyz`	`copy`	new array, every access
`np.asarray(pcd.points)`	`view`	writes reach the point cloud

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/python_facts.json.

2 Voxel downsampling: one average per voxel

```
down = pcd.voxel_down_sample(0.5)
P = np.asarray(down.points)
print(len(xyz), "->", len(P))
```



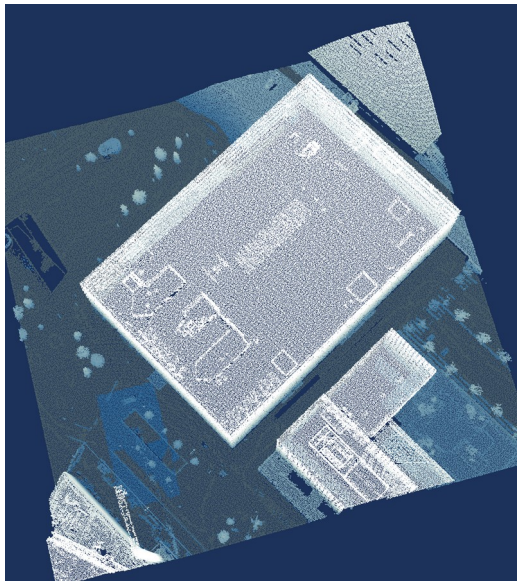


Rule 2: 722 961 points in → 232 180 out at 0.5 m voxels.

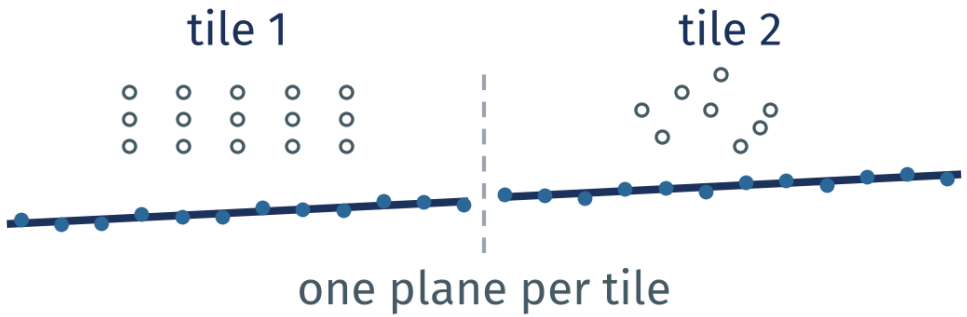
Data: AHN5, Het Waterschapshuis, CC BY 4.0.

3 Ground removal: one plane per tile

```
o3d.utility.random.seed(0)
ground = np.zeros(len(P), bool)
for i in tiles(P, 25):
    z0 = np.percentile(P[i, 2], 2)
    low = i[P[i, 2] < z0 + 1.5]
    t = down.select_by_index(low)
    _, inl = t.segment_plane(
        0.4, 3, 300,
        probability=1.0)
    ground[low[inl]] = z0 < 5
```



Scored against AHN's ground: 98 % precision, 94 % recall.

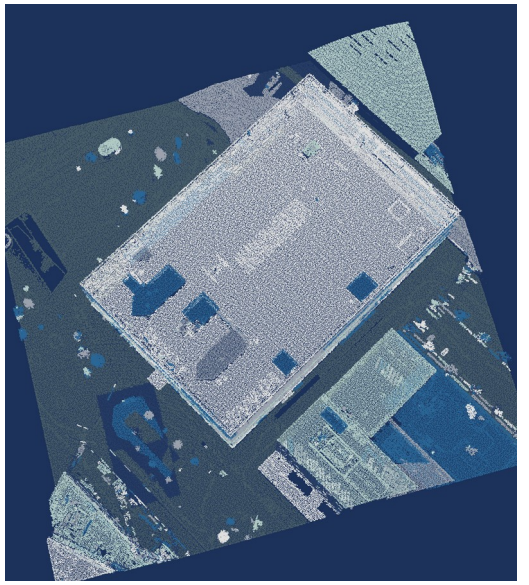


Fit one plane per tile; roofs get none.

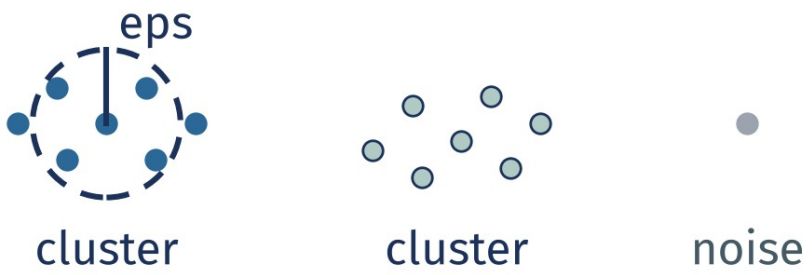
Data: AHN5, Het Waterschapshuis, CC BY 4.0.

4 Clustering: DBSCAN groups points that touch

```
hag = height_above(P, ground)
up = ~ground & (hag > 0.3)
rest = down.select_by_index(
    np.flatnonzero(up))
lab = rest.cluster_dbscan(
    eps=0.75, min_points=6)
ids = np.asarray(lab)
print(ids.max() + 1, "clusters")
```



1 024 clusters.
Clusters are not labels yet.

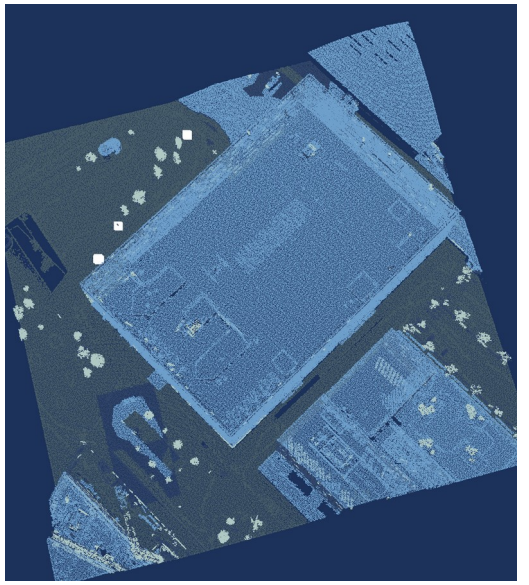


Neighbours within eps make a cluster.

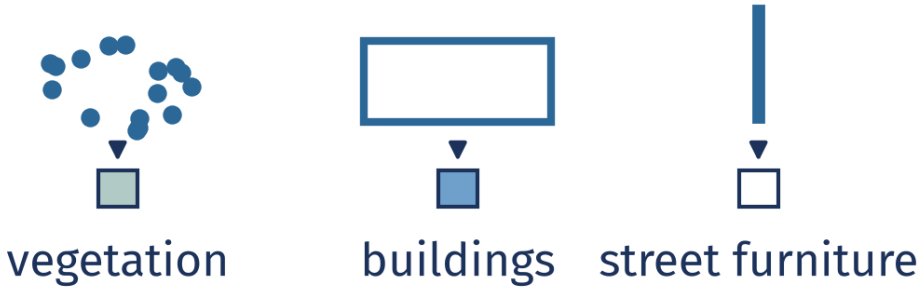
Data: AHN5, Het Waterschapshuis, CC BY 4.0.

5 Labelling: the first rule that fires wins

```python
def name(c):
    if c.multi > 0.4 and c.top > 1:
        return VEGETATION
    if c.foot > 25 and c.top > 2.5:
        return BUILDINGS
    if c.to_building < 2:  # facade
        return BUILDINGS
    if c.slender and c.free:
        return STREET_FURNITURE
    return UNLABELLED
```



Height, footprint and echoes decide the class.

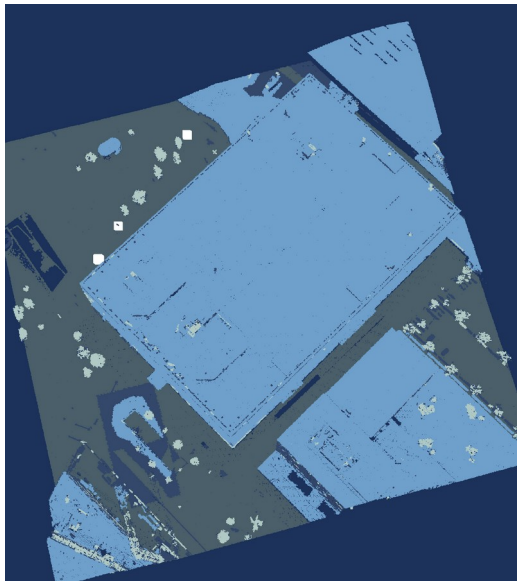


1 024 clusters, not 232 180 points: a Python loop is fine.

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

6 Paint-back: every point takes its voxel's label

```
P = np.asarray(down.points)
_, j = KDTree(P).query(
    xyz, workers=-1)
pl = pv.Plotter(off_screen=True)
pl.add_points(xyz, point_size=3,
    scalars=RGB[cls[j]], rgb=True)
pl.camera_position = CAM
pl.screenshot("f6_final.png")
```



One KDTree query, then PyVista renders it.



each point takes its nearest voxel's colour

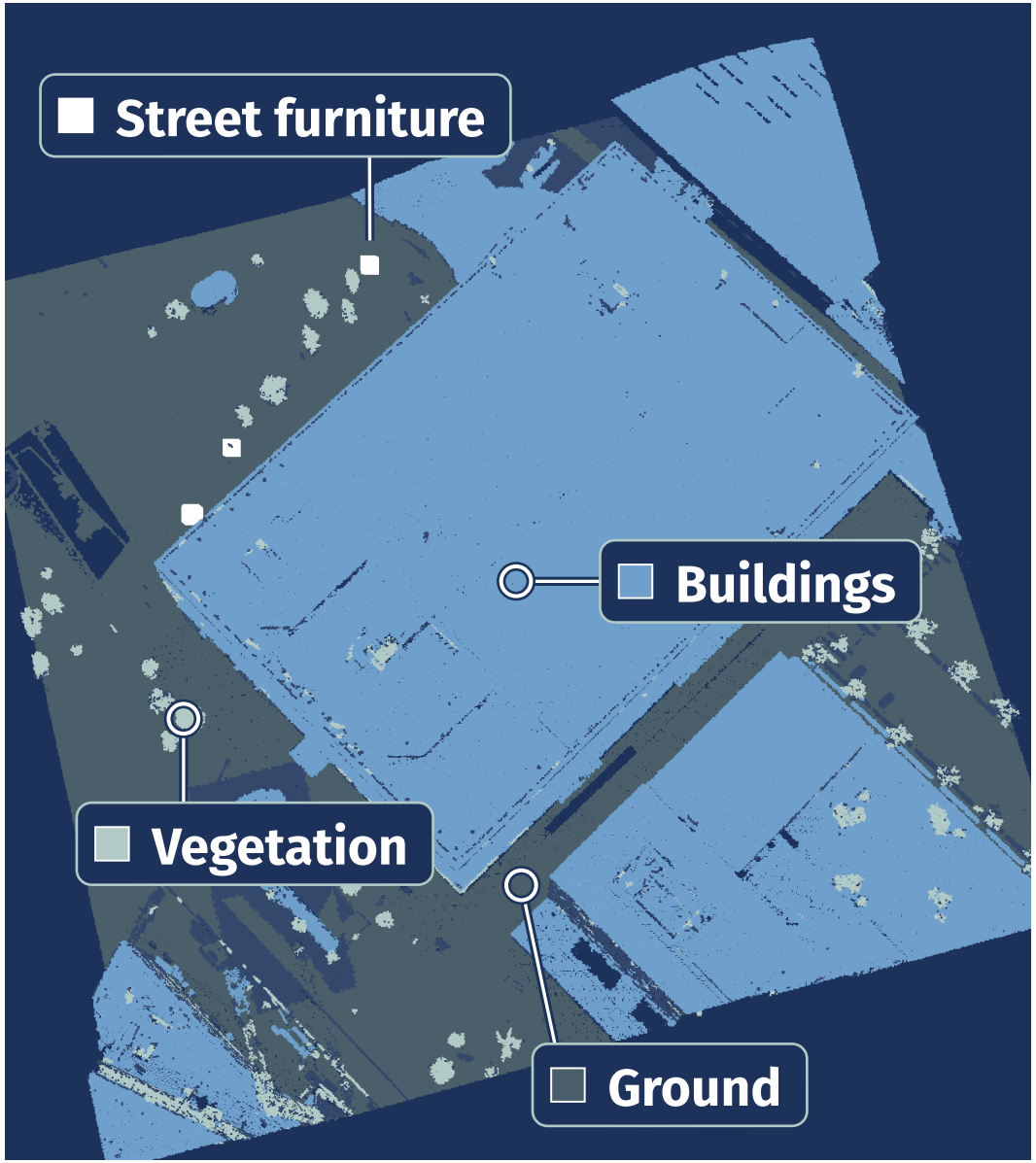
One query labels all 722 961 points.

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

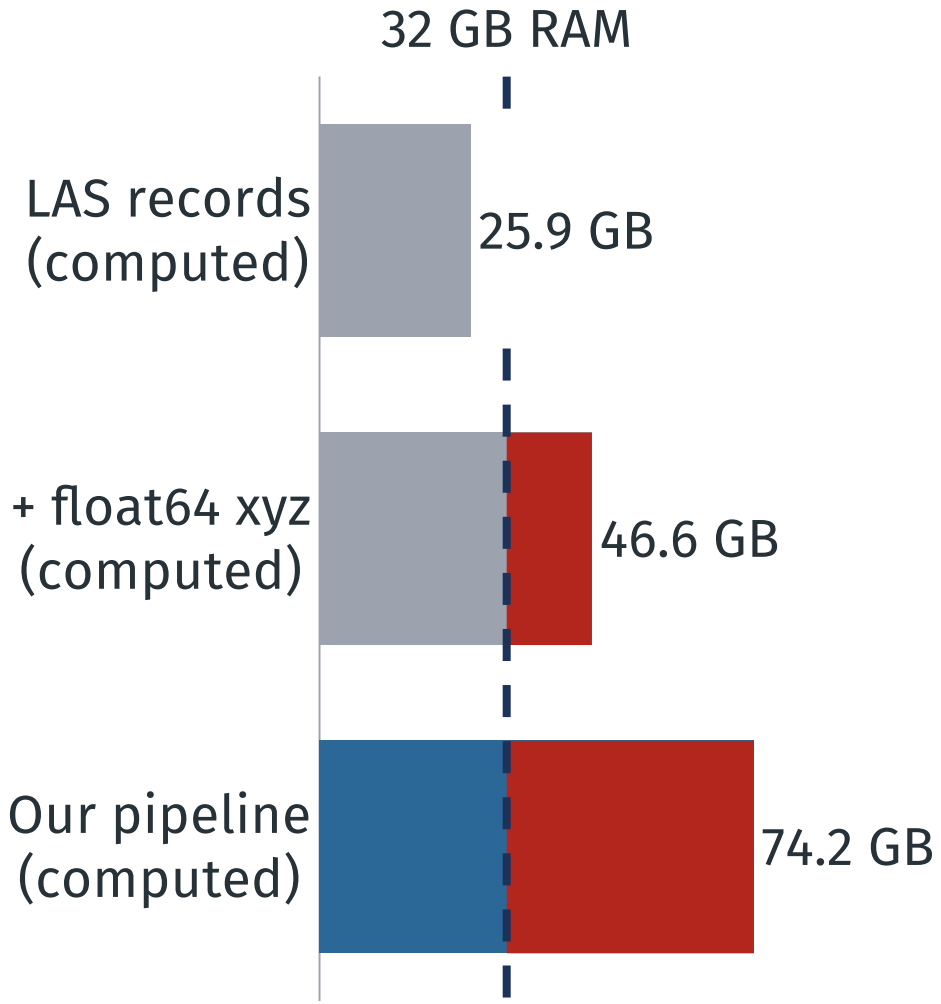
Six steps label every point in 0.6 s

Step	Library	Result	Time
Load	laspy	722 961 points	57 ms
Voxels	Open3D	232 180 voxels	84 ms
Ground	Open3D	98 % precision	81 ms
Clusters	Open3D	1 024 clusters	137 ms
Labels	NumPy	4 classes	151 ms
Paint-back	SciPy	all points	99 ms

No loop over points: the label rules loop over 1 024 clusters.



Data: AHN5, Het Waterschapshuis, CC BY 4.0.



A full AHN sheet would need 74.2 GB

86 B per point in our pipeline

926 203 338 points × 86 B each.

It works on my machine (31.3 GiB RAM).

86 B per point measured on the 1 km AHN tile (21 013 926 points); the 926 203 338 points are the AHN5 sheet 2023_C_31HZ2 that holds the crop, computed, not run. Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 21 013 926 points, median of 3 runs, each in a fresh process; AHN5 1 km file (CC BY 4.0). 1 GB = 2^{30} B. Full: results/memory.json.

Stream chunks, keep the first point per voxel

```
v, keys, pts = 0.5, [], []                          # voxel size: 0.5 m
with laspy.open(path) as f:
    lo, hi = f.header.mins, f.header.maxs           # local origin, extent
    dims = ((hi - lo) // v).astype(np.int64) + 1    # voxels per axis
    for c in f.chunk_iterator(1_000_000):           # a million points
        xyz = np.c_[c.x, c.y, c.z] - lo
        ijk = (xyz // v).astype(np.int64).T         # voxel index per axis
        key = np.ravel_multi_index(ijk, dims)       # one int64 per voxel
        k, first = np.unique(key, return_index=True)
        keys.append(k); pts.append(xyz[first])      # first per voxel
k, first = np.unique(np.concatenate(keys), return_index=True)
points = np.concatenate(pts)[first]                 # earliest chunk wins
```

3.0 GB → 0.8 GB peak memory, 1 km AHN tile

Rule 2: make it small first, then scale up.

Measured on the 1 km AHN tile (21 013 926 points); the full sheet of slide 22 was not run. Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 21 013 926 points, median of 3 runs, each in a fresh process; AHN5 1 km file (CC BY 4.0). 1 GB = 2^{30} B. Full: results/memory.json.

Quiz: which is faster, and by how much?

Distance of every point to the ground plane, 722 961 points.

A

```
a, b, c, d = plane
h = np.empty(len(P))  # distances
for i, p in enumerate(P):
    x, y, z = p[0], p[1], p[2]
    h[i] = abs(a*x + b*y + c*z + d)
```

B

```
a, b, c, d = plane
x, y, z = P.T  # columns, no copy
h = np.abs(a*x + b*y + c*z + d)
```

The faster one wins by about: 2× 10× 100× 1000×

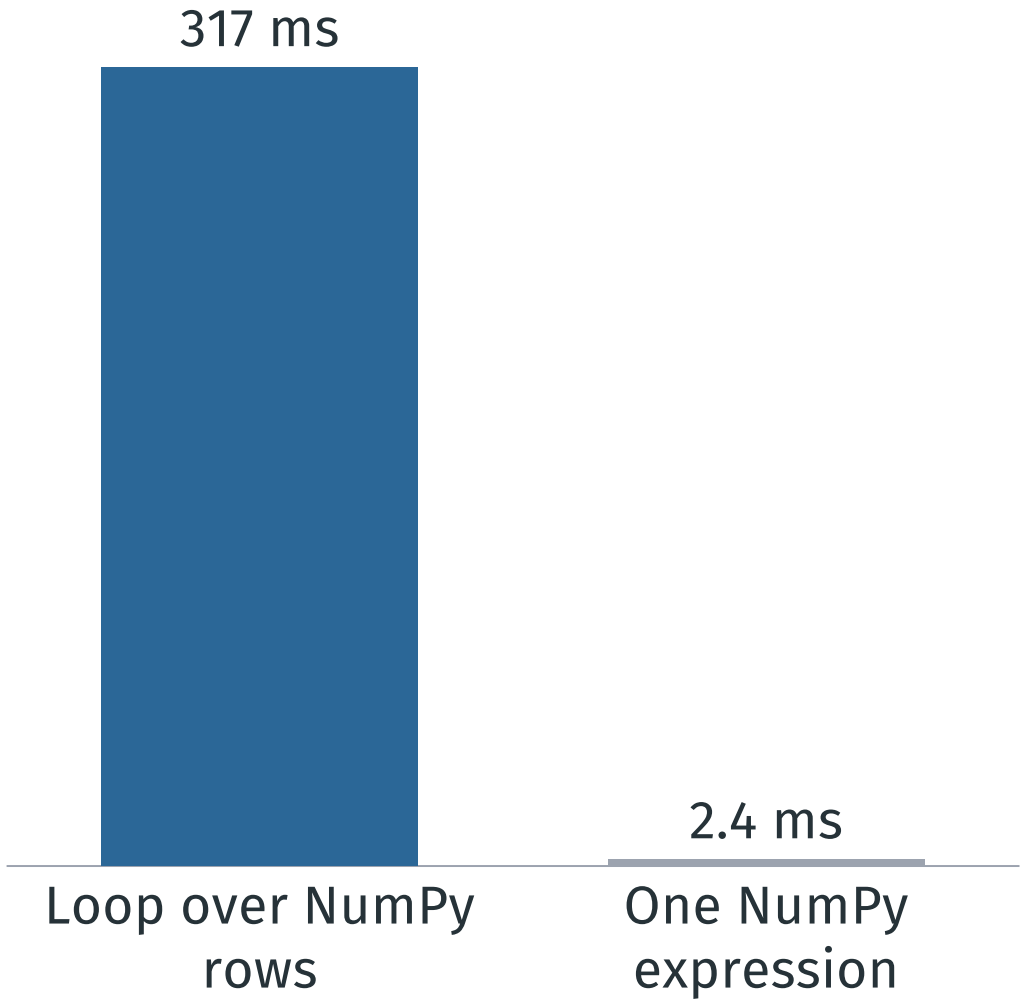
Both give identical results (np.array_equal). P: the crop's xyz from slide 14, column-ordered. plane: one seeded segment_plane on the whole crop.

How much slower is the loop over rows?

130× slower

Loop: 12 calls × 722 961 rows = 8.7 million calls

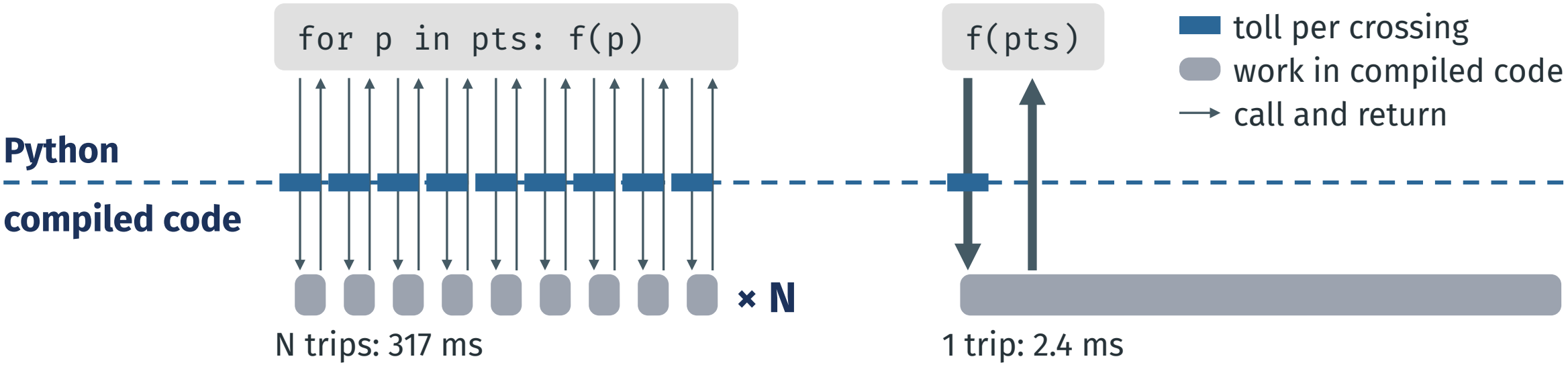
One expression: 7 calls in total



Same data, same answer, far fewer calls into compiled code.

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_cliff.json.

Every trip into compiled code pays a toll



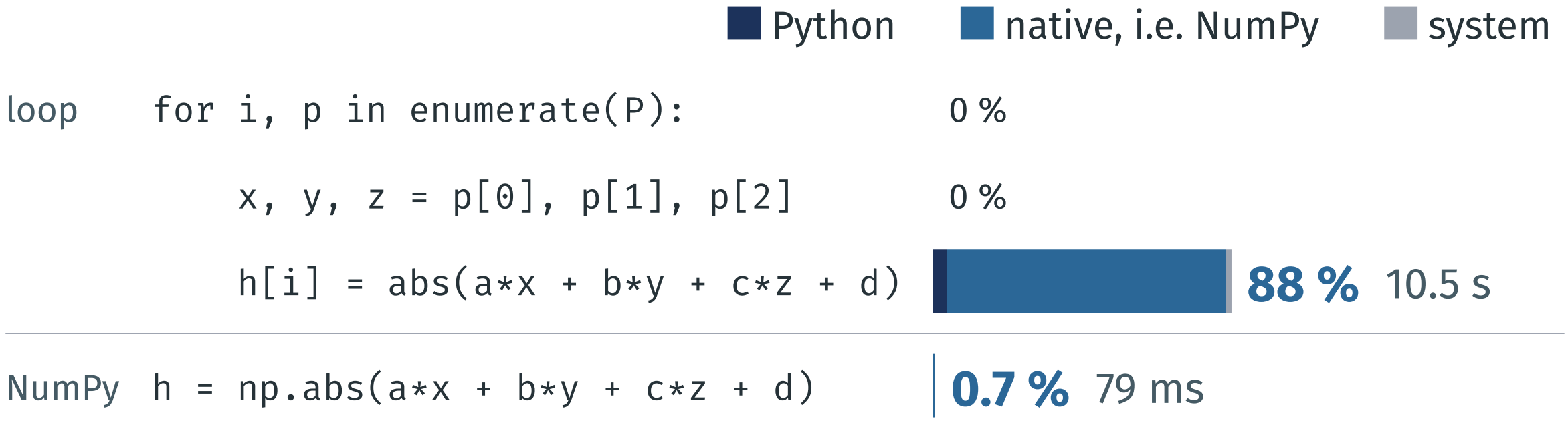
439 ns per loop step

Rule 3: one call on the whole array, not N calls on single points.

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_cliff.json.

Profile before you vectorise

```
$ scalene run --cpu-only loop.py
```



Different setup: under the profiler, each version runs 30 times; slide 25 holds the baseline timings. Conditions: Ryzen 7 5700X, Python 3.13.15, Scalene 2.3.0 (--cpu-only, a sample every 1 ms); 722 961 points, both versions 30 times per run, median of 3 runs; AHN5 crop (CC BY 4.0). Shares are of the whole run, imports and reading the file included; times are share × the 11.9 s run. Full: results/profile_scalene.json.

Will free-threaded Python save my loop?

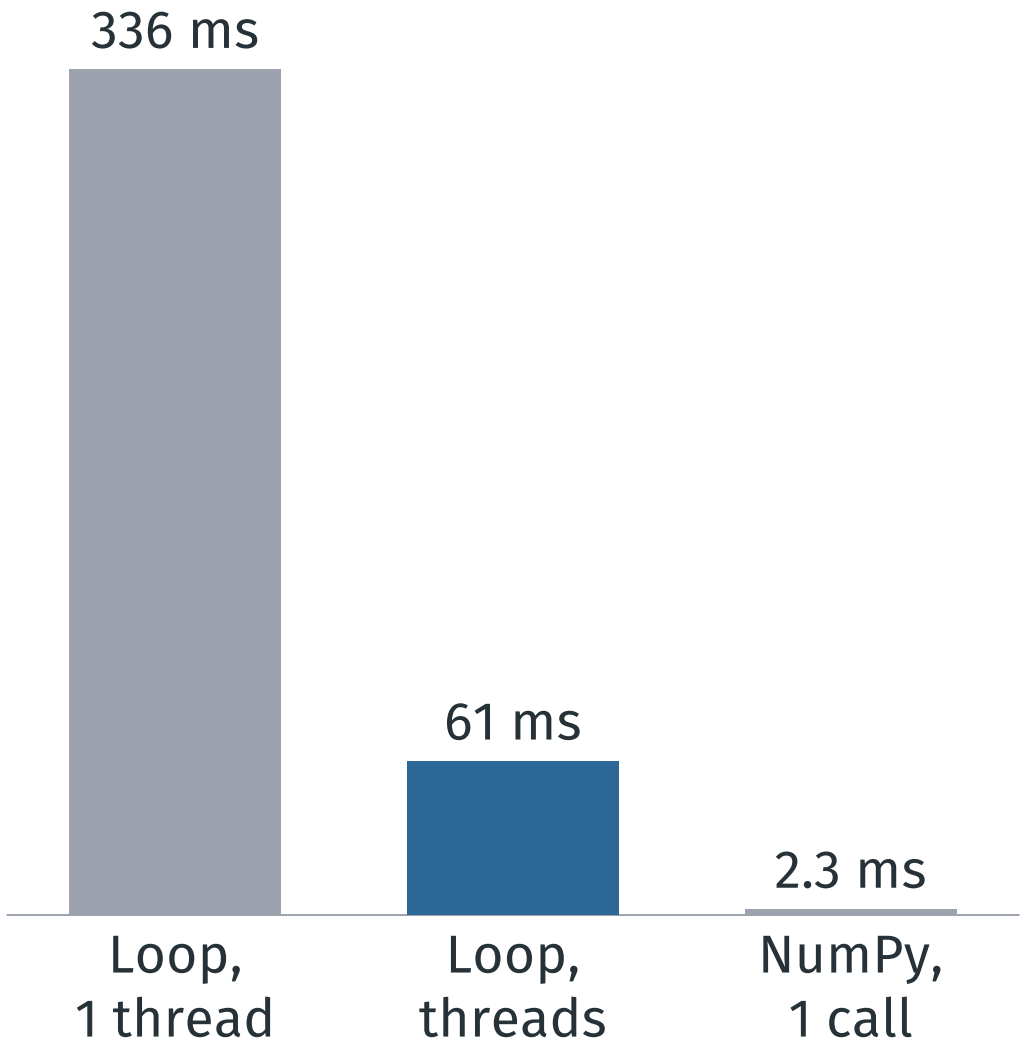
5.5× faster on 8 free threads

Loop: 12 calls × 722 961 rows = 8.7 million calls

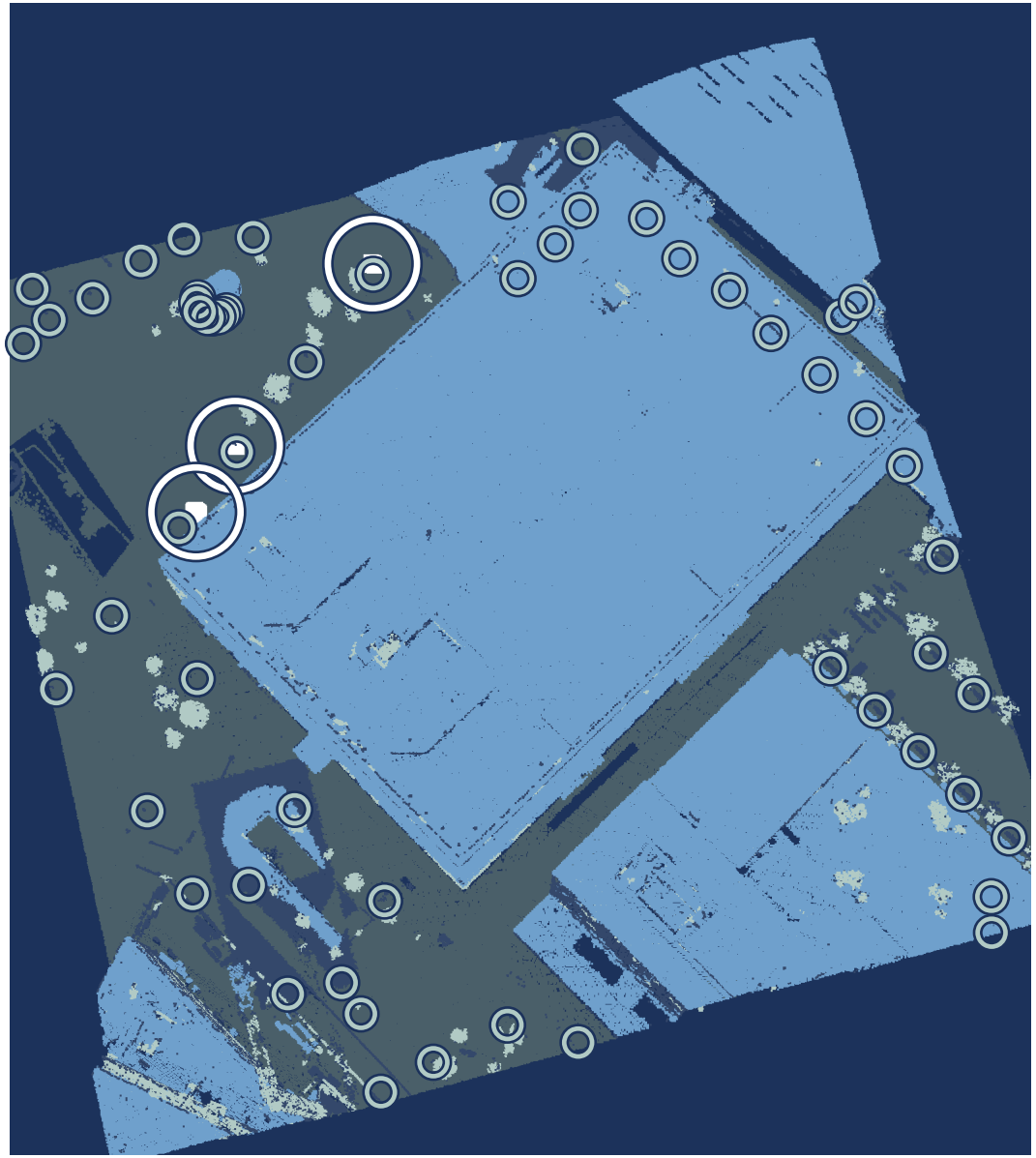
8 threads: same calls, 1.1 million each

NumPy: 7 calls in total

Rule 3: one NumPy call still beats 8 threads by 27×; GIL threads gain nothing.



Different setup: Python 3.14 builds. Loop 336 ms and NumPy 2.3 ms here; 317 ms and 2.4 ms on 3.13 (slide 25). Conditions: Ryzen 7 5700X, CPython 3.14.7 free-threaded and 3.14.7, NumPy 2.5.4; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_freethreaded.json.



The register has 61 poles; we found 3

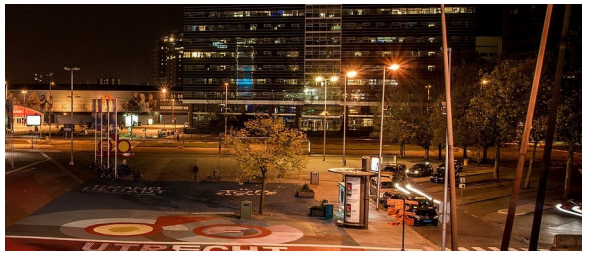
Expectation: the BGT register

61 poles 51 lamp posts, 10 bollards

Reality: our pipeline

3 objects 2 of them on a lamp post



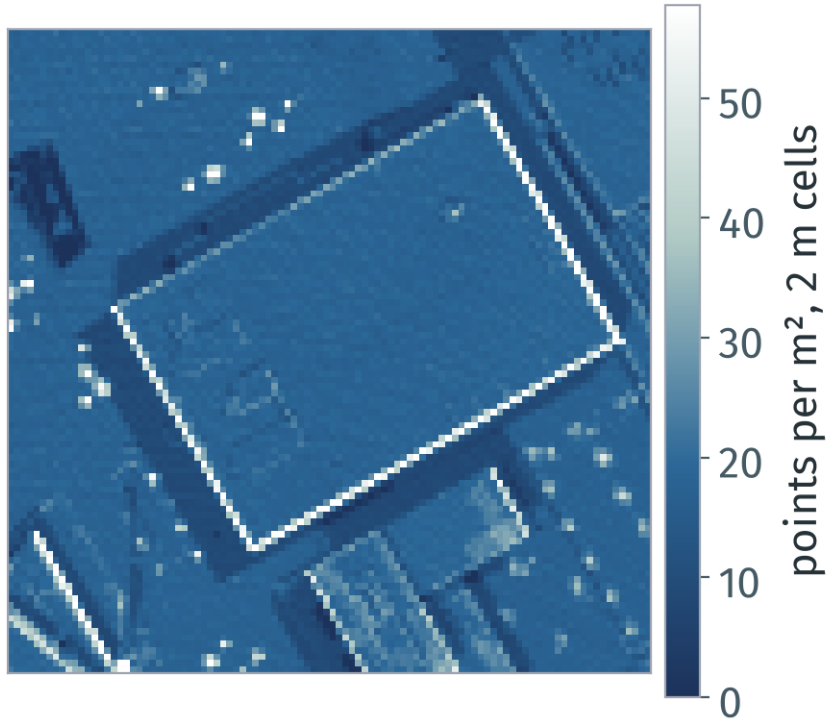


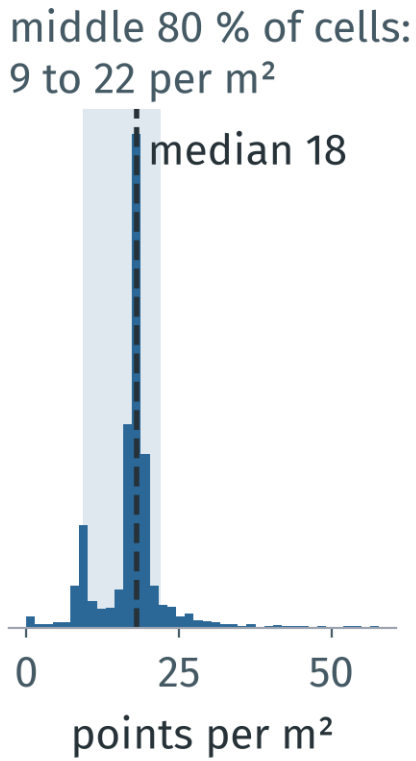
42 of 61 poles get no LiDAR point above 1.5 m.

Poles: BGT, Kadaster, CC BY 4.0. Photos: Wikimedia Commons, CC BY-SA 4.0 and CC BY 3.0.

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

From the air, the city is sampled unevenly





Rule 4: density, noise and gaps vary within one file. Measure them before tuning anything.

- Here the middle 80 % of cells hold 9 to 22 points per m²; median 18.
- Occlusion, flight-strip overlap, angle and material decide where points land.
- Every point carries error: AHN4 specifies at most 5 cm in height; AHN5 publishes no figure yet.

Four rules, each backed by this talk's numbers

1 **Think in arrays, not points**

24 B per point per xyz copy

slides 14, 15

2 **Make it small first**

3.0 GB → 0.8 GB peak

slides 16, 17, 19, 22, 23

3 **Let compiled code loop**

130× faster than a loop

slides 24 to 28

4 **Look before you compute**

61 poles on the register, 3 found

slides 29, 30

What this doesn't do yet, and what's next

A baseline in plain Python, not production accuracy.

Limits

- 3 of 61 poles found
- Vegetation: not scored
- Buildings: 83 % precision, a lower bound
- One 200 m crop, one timed run

Next

- Ground and clusters on tiles with overlap (B2)
- A classifier for the labels (B5)
- Your own point cloud (B6)

Thank you

Write the plan in Python; let compiled code loop.



Try it on your own AHN tile.

Sergio Peral
sergioperalgarijo99@gmail.com
github.com/Sloppy1999

Data: AHN5, Het Waterschapshuis, CC BY 4.0.
Licence: creativecommons.org/licenses/by/4.0
Title shared with F. Poux, 3D Data Science with Python, O'Reilly 2025; this talk is independent.